

Implement Client Script & UI Policy (Incident)

Project Phase Document

1. Ideation Phase

Problem Statement

Incident records require consistent and accurate data entry. Manual checks may lead to incomplete, inconsistent, or incorrect data, affecting reporting, SLA compliance, and service quality. The project addresses this by applying conditional field behavior and validation at the user-interface level.

Project Idea

- Use ServiceNow UI Policies and Client Scripts on Incident forms.
- Make fields mandatory based on Impact.
- Automatically set Urgency for High Impact incidents.
- Prevent invalid save and block State changes through list editing.

Objective

Demonstrate client-side controls that enforce data integrity, automate field updates, control field behavior, and prevent incorrect Incident submission.

Skills

- Incident Management
- UI Policy and UI Policy Actions
- Client Scripts
- Form Validation

2. Requirement Analysis

The system must apply rules to Incident records based on the Impact field and validate required information before saving.

2. Requirement Analysis – Detailed

Inputs

- Impact
- Assigned To
- Urgency
- State

Rules

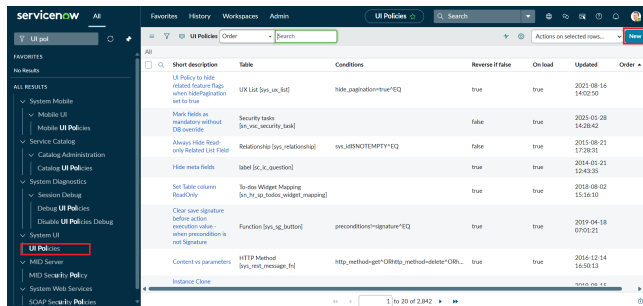
- Impact = 1 – High triggers High Impact Control.
- Assignment group is mandatory under the UI Policy.
- Urgency becomes read-only when Impact is High.
- High Impact + empty Assigned To must not be saved.
- State cannot be changed using list editing.

Validation

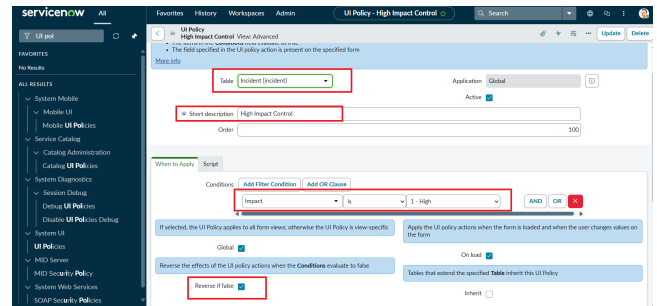
The configured behaviors are verified through mandatory enforcement, successful save, reverse condition, list-edit blocking and form-based update tests.

3. Project Design Phase

UI Policy design – High Impact Control



Short description	Table	Conditions	Reverse if false	On load	Updated	Order
UI Policy to hide related buttons flags when highRegistration set to true	UX List [sys_ui_list]	high_registration=trueEQ	true	true	2021-08-16 14:02:50	
Mark fields as mandatory without O2 override	Security tasks [sys_sec_task]		false	true	2025-05-28 14:28:42	
Always Hide Read-only Related List Page	Relationship [sys_relationship]	sys_HRISNOTEMPTYEQ	false	true	2015-08-21 17:28:51	
Hide meta fields	label [sys_label]		true	true	2024-05-21 12:42:35	
Set table column Read/Write	To-do Widget Mapping [sys_to_do_widget_mapping]		true	true	2018-08-02 15:16:50	
Clear save signature before action execution value when precondition is not Signature	Function [sys_func_button]	precondition=signatureEQ	true	true	2019-04-18 07:02:21	
Content vs parameter	HTTP Method [sys_rest_message_14]	http_method=getORhttp_method=deleteOR...	true	true	2026-12-14 16:50:13	
Balance Case						



UI Policy - High Impact Control

Table: Incident [Incident]

Application: Global

Active: ☒

Order: 100

When to Apply: Script

Conditions: Add Filter Condition | Add OR Clause

Impact is 1 - High

AND OR

Reverse if false: ☒

On load: ☒

inherit: ☐

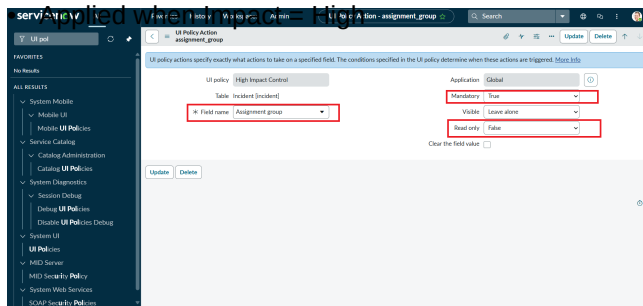
UI Policy Configuration

- Name: High Impact Control
- Table: Incident
- Active: true
- Condition: Impact is 1 – High
- UI Policy Action: Assignment group
- Mandatory: enabled
- Reverse if false: true

3. Project Design Phase – UI Policy Action

Urgency Field Control

- Field name: Urgency
- Read-only: true
- Visible: unchanged



Expected Behavior

When the High Impact Control condition is met, Urgency remains visible but cannot be edited. When the condition becomes false, the policy action is reversed.

3. Project Design Phase – onChange Client Script

Configuration

Name: Auto set urgency for high impact

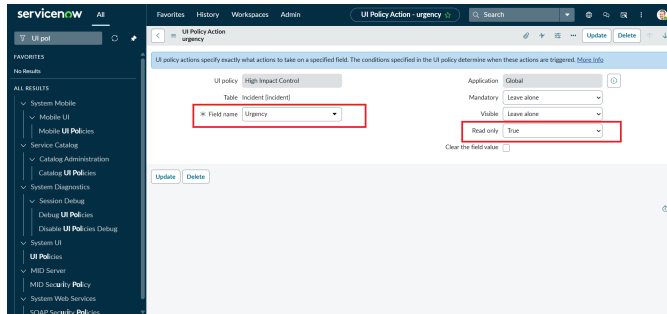
Table: Incident

Type: onChange

Field: Impact

Active: true

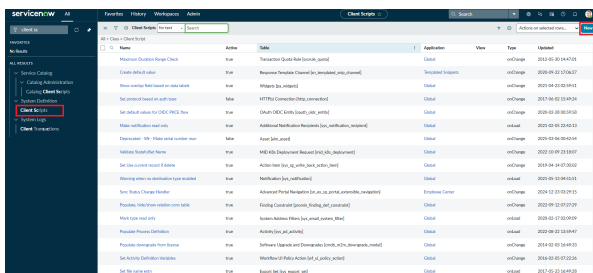
Action: set Urgency to 1 – High



```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == '') return;  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

Expected: changing Impact to High automatically updates Urgency and shows an information message.

3. Project Design Phase – onSubmit Client Script



Configuration

Name: Prevent save if Assigned To missing

Table: Incident

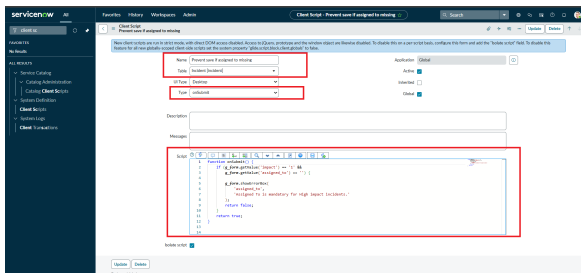
Type: onSubmit

Active: true

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {
        g_form.showErrorMessage(
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

Expected: High Impact incidents cannot be saved when Assigned To is empty.

3. Project Design Phase – onCellEdit Client Script



Configuration

Name: Prevent state change via list edit

Table: Incident

Type: onCellEdit

Field: State

Active: true

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.');
```

Expected: direct State changes from the list are blocked; form-based State updates remain available.

4. Project Planning Phase

Planning	Define the Incident data rules and expected field behavior.
Configuration	Create the UI Policy and UI Policy Action.
Scripting	Implement onChange, onSubmit and onCellEdit Client Scripts.
Testing	Run mandatory, successful-save, reverse-condition, list-edit and form-update tests.
Demonstration	Show the configured Incident form and the expected results.

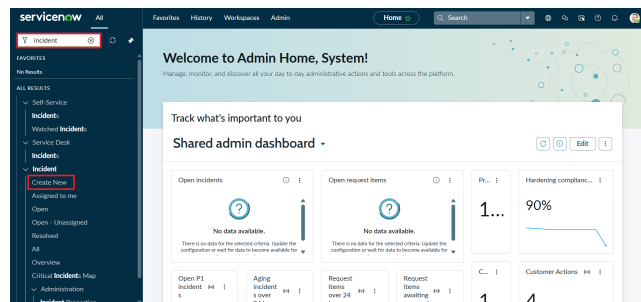
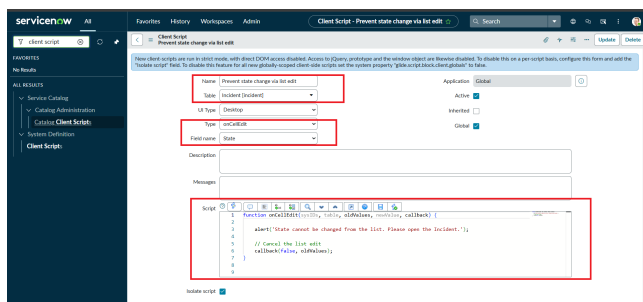
Implementation Sequence

- UI Policy → High Impact Control
- UI Policy Action → Urgency read-only
- onChange → auto-set Urgency
- onSubmit → validate Assigned To
- onCellEdit → block State list editing
- Testing → verify all configured behaviors

5. Project Document Phase

Testing Activity 1 – Mandatory Enforcement

- Incident → Create New
- Set Impact = High
- Leave Assigned To empty
- Click Submit
- Expected: record is not saved and an error appears



Testing Activity 2 – Successful Save

- Fill Assigned To
- Click Submit
- Expected: record saves successfully
- Verify Urgency auto-setting, read-only behavior and State restriction

6. Project Documentation – Test Results

Activity 3 – Reverse Condition Test

- Open an existing High Impact Incident.
- Change Impact to Medium.
- Assigned To should no longer be mandatory.
- Urgency should become editable.
- Save successfully.

Activity 4 – List Edit Blocking

- Incident → All.
- Double-click State.
- Alert should appear.
- State value should remain unchanged.

Activity 5 – Form-Based Update

- Open the Incident form.
- Change State.
- Click Update.
- State change should save successfully.

7. Project Demonstration

This screenshot shows the 'Incident - Create' form in ServiceNow. The form is for incident INCD000012. The 'Impact' dropdown is set to 'None' and is highlighted with a red box. The 'Urgency' dropdown is set to 'High' and is also highlighted with a red box. The 'Assigned to' field is set to 'Beth Angulo' and is highlighted with a red box. The 'State' dropdown is set to 'New'. The 'Channel' dropdown is set to 'None'. The 'Category' dropdown is set to 'Incident / Help'. The 'Subcategory' dropdown is set to 'None'. The 'Service' dropdown is set to 'None'. The 'Service offering' dropdown is set to 'None'. The 'Configuration item' dropdown is set to 'testing UI policy'. The 'Short description' field contains 'testing UI policy'. The 'Description' field is empty. The 'Related search' field contains 'testing UI policy'. The 'Knowledge & Catalog ID' dropdown is set to 'None'. The 'Create' button is at the bottom right.

This screenshot shows the 'Incident - INCD000012' form in ServiceNow. The form is for incident INCD000012. The 'Impact' dropdown is set to 'Medium' and is highlighted with a red box. The 'Urgency' dropdown is set to 'High' and is also highlighted with a red box. The 'Assigned to' field is set to 'Beth Angulo' and is highlighted with a red box. The 'State' dropdown is set to 'None'. The 'Channel' dropdown is set to 'None'. The 'Category' dropdown is set to 'Incident / Help'. The 'Subcategory' dropdown is set to 'None'. The 'Service' dropdown is set to 'None'. The 'Service offering' dropdown is set to 'None'. The 'Configuration item' dropdown is set to 'testing UI policy'. The 'Short description' field contains 'testing UI policy'. The 'Description' field is empty. The 'Related search' field contains 'testing UI policy'. The 'Knowledge & Catalog ID' dropdown is set to 'None'. The 'Create' button is at the bottom right.

- Demonstrate High Impact UI Policy behavior.
- Show Urgency becoming read-only when Impact is High.
- Change Impact to High and show automatic Urgency update.
- Try saving without Assigned To and show validation.
- Try State list editing and show the alert.
- Change State through the form and show successful update.

Conclusion

The Implement Client Script & UI Policy (Incident) micro project demonstrates how UI Policies and Client Scripts work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. The onChange, onSubmit and onCellEdit scripts, together with UI Policy actions, control Assigned To, Urgency and State behavior according to the configured conditions.

Key Outcome

- Conditional field behavior is enforced at the UI level.
- Urgency can be automated for High Impact incidents.
- Required information is validated before saving.
- Restricted list editing is blocked while valid form updates remain possible.